Software Requirements Specification (SRS) Template

Project: queue based traffic flow simulator
Version: 1.0
Authors: Harini, Neal
Date: 07-09-26
Status: Model Answer / Approved

Revision history

Version	Date	Author	Change summary	Approval
1.0	18-08-2025	Neal, Harini	SRS with diagrams embedded	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Dr. Sheela Devi		

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

The purpose of the Traffic Management System (Simulation) is to simulate and analyze the flow of vehicles at a traffic intersection using the C programming language and fundamental data structures, primarily queues. The system represents vehicles as elements in queues and manages their movement based on traffic signal conditions.

The simulation helps demonstrate how data structures can be applied to a real-world traffic management problem. It allows users to observe vehicle arrivals, waiting queues, signal changes, and vehicles passing through an intersection. The system can also provide basic traffic statistics such as queue length, number of vehicles processed, and waiting time.

The project is intended as a simulation system for educational and analytical purposes and does not directly control real-world traffic signals.

1.2 Scope

The Traffic Management System will simulate traffic flow at a road intersection. Vehicles arriving at different directions of the intersection will be placed into separate queues. The traffic signal will determine which direction is allowed to move at a given time.

The system will provide the following functionalities:

- Simulate the arrival of vehicles at an intersection.
- Maintain separate queues for different traffic directions.
- Add vehicles to queues using the enqueue operation.
- Remove vehicles from queues using the dequeue operation when they pass through the intersection.
- Simulate traffic signals such as Red, Yellow, and Green.
- Control vehicle movement based on the current traffic signal.
- Display the current number of waiting vehicles.
- Display queue lengths for different directions.
- Allow different traffic densities to be simulated.
- Allow traffic signal timings to be configured.
- Calculate basic traffic statistics.
- Allow the user to start, pause/resume, and stop the simulation.

The system will be developed in C using data structures such as queues, structures, arrays, and linked lists where required.

The project is limited to traffic simulation and will not interact with real traffic signals, cameras, sensors, GPS systems, or physical vehicles.

1.3 Audience

Project Developers: To understand the functional and non-functional requirements and implement the system accordingly.

Project Guide/Faculty: To evaluate the system requirements, design, and implementation.
Testers: To understand the expected system behavior and develop test cases.
Students: To understand the application of data structures such as queues in a real-world simulation.
End Users: To understand how the traffic simulation can be operated and monitored.

1.4 Definitions

List of acronyms:

Acronym	Full Form
TMS	Traffic Management System
SRS	Software Requirements Specification
UML	Unified Modeling Language
FIFO	First-In-First-Out
UI	User Interface
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
C	C Programming Language

.

2. Overall description

2.1 Product perspective

It is a model to simulate the vehicle behavior at intersections using

queue-based data structures, run entirely on a single machine with no additional hardware requirements beyond a standard computer.

2.2 Major product functions (detailed)

Generate vehicle arrivals at controllable rates
Queuing vehicles at intersection signals, FIFO queue in the lane or direction.
Simulating the traffic signal transitions (red/green/yellow) on a timer
Dequeue and move vehicles through the intersection on green signal
Track and report queue length, average wait time, and throughput over the simulation run
Allow user control to the simulation parameters (rate of arrival, time for each signal, number of lanes)
Being able to reset and re-run simulations with different parameter sets
Represent the simulation (console output, log file, or basic visualization)

2.3 Operating environment

Runs on a standard laptop OS (Windows) executed via a standard runtime environment for C. No network connectivity required, simulation runs locally.

2.4 Constraints

Single-threaded execution, simulation operates on simplified assumptions (uniform vehicle behavior, no real-world sensor or GPS data); no real-time hardware integration; queue capacity can be configured.

3. External interface requirements

3.1 User interfaces

The Traffic Management System will provide a console based user interface developed using the C programming language. The user will interact with the system through the keyboard and view the simulation output on the screen.

The user interface shall provide the following:

- A main menu for accessing different functions of the system.
- Options to start, pause, resume, and stop the simulation.
- Options to configure traffic density and traffic signal timings.
- Display of the current traffic signal status.
- Display of vehicles waiting in each direction.
- Display of the queue length for each direction.
- Display of vehicles that have passed through the intersection.
- Display of traffic statistics after or during the simulation.
- Clear prompts for entering values.
- Appropriate error messages for invalid inputs.

3.2 Hardware interfaces

The system does not require any specialized hardware because it is a software-based traffic simulation.

The system shall operate on a standard computer or laptop with the following hardware interfaces:

- Keyboard: Used to enter menu selections and configuration values.
- Monitor/Display: Used to display the traffic simulation, queue status, signal status, and statistics.
- Processor: Used to execute the simulation and perform queue operations.
- RAM: Used to store vehicle and queue information during execution.
- Storage: Used to store the executable program and, if implemented, simulation results.

The system will not directly interface with:

- Physical traffic lights.
- Traffic cameras.
- Road sensors.
- GPS devices.
- Vehicle tracking devices.
- IoT traffic-control devices.

These may be considered as future extensions of the system.

3.3 Software interfaces

The Traffic Management System will interact with the following software components:

Operating System

The application shall run on commonly used operating systems such as:

- Windows
- Linux
- macOS

C Compiler

The system shall be developed using the C programming language and compiled using a standard C compiler such as GCC.

C Standard Libraries

The implementation may use standard C libraries such as:

- stdio.h – Input and output operations.
- stdlib.h – Memory allocation and general utilities.
- time.h – Random number generation and simulation timing.
- string.h – String manipulation, if required.

Data Structures

The system will use C-based data structures to represent traffic conditions, primarily:

- Queues for waiting vehicles.
- Structures for storing vehicle information.

- Arrays or linked lists where required.

The system shall not require external databases or third-party software for its basic operation.

3.4 Communications

The basic version of the Traffic Management System is a standalone application and does not require communication over a network.

Therefore:

- The system does not require an Internet connection.
- The system does not communicate with external servers.
- Vehicle information is generated and processed locally.
- Traffic signals and queues are simulated within the application.
- User input is received through the local keyboard.
- Simulation output is displayed on the local monitor.

If the system is extended in the future, communication interfaces could be added for receiving real-time traffic information from external sensors, databases, or IoT devices. Such communication is outside the scope of the current project.

4. System features (detailed)

4.1 Authentication

Description: Generate simulated vehicles arriving at intersections according to a configurable arrival pattern

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-001	The system shall generate vehicles at intervals determined by a configurable arrival rate (e.g., Poisson distribution).	Functional	High	Simulation Design	AC-TFS-F-001: Given arrival rate λ, vehicle count over N time steps matches expected distribution within tolerance. Test: TC-Gen-01	Requires random number generator module
TFS-F-002	The system shall assign each generated vehicle a unique identifier and creation	Functional	Medium	Developer	AC-TFS-F-002: Every vehicle object has a unique ID and non-null timestamp. Test: TC-Gen-02	Used for wait-time calculation

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	timestamp (simulation time).					

4.2 Queue Management

Description: Manage vehicles waiting at an intersection using FIFO queue behavior per lane/direction.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-010	The system shall enqueue a vehicle when it arrives at an intersection with a red or yellow signal.	Functional	High	Simulation Design	AC-TFS-F-010: Vehicle appears in queue data structure immediately upon arrival during non-green signal. Test: TC-Queue-01	Depends on Signal Control (4.3)
TFS-F-011	The system shall dequeue vehicles in FIFO order when the signal turns green.	Functional	High	Simulation Design	AC-TFS-F-011: First vehicle enqueued is the first dequeued; order preserved across all dequeues. Test: TC-Queue-02	Core correctness requirement
TFS-F-012	The system shall enforce a maximum queue	Functional	Medium	Developer	AC-TFS-F-012: When queue is at max capacity,	Optional feature; must document chosen

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	capacity (if configured) and handle overflow according to a defined policy (e.g., reject or block new arrivals).				new vehicle is handled per policy without data corruption. Test: TC-Queue-03	

4.2 Cash Withdrawal

Description: Dispense cash in validated denominations and ensure ledger consistency with bank core.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ATM-F-010	The system shall verify sufficient available balance and daily limit before dispensing cash.	Functional	High	Business	AC-ATM-F-010: Dispense occurs only if balance >= amount and within limits. Test: TC-WD-01	Core API check required
ATM-F-011	The system shall dispense requested amount using available denomin	Functional	High	Operations	AC-ATM-F-011: Dispensed amount equals requested; cassette counts	Hardware interaction with dispenser

	ations and update cash cassette counts.				decremented. Test: TC-WD-02	

4. System Features (detailed)

4.1 Vehicle Generation

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-001	The system shall generate vehicles at intervals determined by a controllable arrival rate (e.g., Poisson distribution).	Functional	High	Simulation Design	AC-001: Vehicle count over N steps matches expected distribution within tolerance. Test: TC-Gen-01	Requires RNG module
TFS-F-002	The system shall assign each vehicle a unique ID and creation timestamp (simulation time).	Functional	Medium	Developer	AC-002: Every vehicle has unique ID and non-null timestamp. Test: TC-Gen-02	Used for wait-time calc

4.2 Queue Management

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-010	The system shall enqueue a vehicle when it arrives during a red or yellow signal.	Functional	High	Simulation Design	AC-010: Vehicle appears in queue immediately on arrival during non-green. Test: TC-Queue-01	Depends on 4.3
TFS-F-011	The system shall dequeue vehicles in FIFO order on green signal.	Functional	High	Simulation Design	AC-011: First enqueued is first dequeued, order preserved. Test: TC-Queue-02	Core correctness
TFS-F-012	The system shall enforce a maximum queue capacity (if configured) and handle overflow per a defined policy.	Functional	Medium	Developer	AC-012: At max capacity, new arrivals handled per policy, no data corruption. Test: TC-Queue-03	Document chosen policy
TFS-F-013	The system shall support a separate queue per lane/direction at each intersection.	Functional	Medium	Simulation Design	AC-013: Vehicles in different lanes tracked independently with no cross-contaminati	Needed for multi-lane realism

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
					on. Test: TC-Queue-04	

4.3 Signal Control

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-020	The system shall cycle each signal through red, green, and (optionally) yellow states per configurable durations.	Functional	High	Simulation Design	AC-020: State changes match configured durations within one time-step tolerance. Test: TC-Sig-01	Timing config required
TFS-F-021	The system shall support independent signal timing per intersection (multi-intersection setups).	Functional	Low	Developer	AC-021: Each intersection's cycle configured/operates independently. Test: TC-Sig-02	Multi-intersection only

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-022	The system shall log every signal state change with its simulation timestamp.	Functional	Medium	Evaluator	AC-022: Log contains an entry for every transition with correct timestamp. Test: TC-Sig-03	Feeds into reporting/audit

4.4 Simulation Reporting

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-030	The system shall record each vehicle's wait time and compute average wait time at run end.	Functional	High	Evaluator	AC-030: Reported average matches manual calculation for known scenario. Test: TC-Rep-01	Depends on 4.2 timestamps
TFS-F-031	The system shall record the maximum queue length observed during the run.	Functional	Medium	Evaluator	AC-031: Reported max matches true observed peak. Test: TC-Rep-02	—
TFS-F-032	The system shall output a summary report at run end	Functional	Medium	Student/Operator	AC-032: Report generated with all required	Format decided in design

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	(console or file).				metrics. Test: TC-Rep-03	

4.5 Simulation Configuration & Control

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-F-040	The system shall allow configuring arrival rate, signal timings, and number of lanes/intersections before a run.	Functional	High	Student/Operator	AC-040: Configured values applied correctly at start. Test: TC-Cfg-01	UI/CLI input TBD
TFS-F-041	The system shall allow resetting and re-running with new parameters without restarting the application.	Functional	Low	Developer	AC-041: State fully resets; no residual data. Test: TC-Cfg-02	—
TFS-F-042	The system shall allow saving the current configuration to a file for reuse in future runs.	Functional	Low	Student/Operator	AC-042: Saved file reproduces identical configuration when reloaded. Test: TC-Cfg-03	Supports repeatability

5.1. Security

5.1.1 Security Objectives

SEC-OBJ-1 — Integrity of simulation data: Ensure configuration files, logs, and reports cannot be silently altered, preserving trust in simulation results.

SEC-OBJ-2 — Controlled access to configuration/admin functions: Ensure only the intended operator can modify simulation configuration, particularly in any shared or multi-user deployment.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TFS-S-001	The system shall restrict write access to configuration files to the authorized local user/process.	Security	Medium	Developer	AC-S001: Unauthorized process/user cannot modify config file (verified via file permission test). Test: TC-Sec-01	Relies on OS file permissions
TFS-S-002	The system shall validate and sanitize all user-supplied configuration inputs before use.	Security	High	Developer	AC-S002: Invalid/out-of-range/malformed inputs are rejected with a clear error, no crash. Test: TC-Sec-02	Prevents malformed-input crashes
TFS-S-003	The system shall treat generated logs/reports as append-only during a run, preventing in-place	Security	Medium	Evaluator	AC-S003: Attempting to alter a written log entry during the run is prevented or detected. Test: TC-Sec-03	Supports result integrity

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	modification of prior entries.					
TFS-S-004	The system shall record a timestamped audit entry whenever configuration parameters are changed.	Security	Medium	Evaluator	AC-S004: Every config change produces a corresponding audit log entry. Test: TC-Sec-04	Supports SEC-OBJ-1
TFS-S-005	The system shall handle corrupted or malformed configuration/input files gracefully, without exposing internal file paths or stack traces to the user.	Security	Medium	Developer	AC-S005: Malformed file triggers a generic user-facing error; no internal details leaked. Test: TC-Sec-05	Basic hardening

6. Quality attributes & Acceptance tests

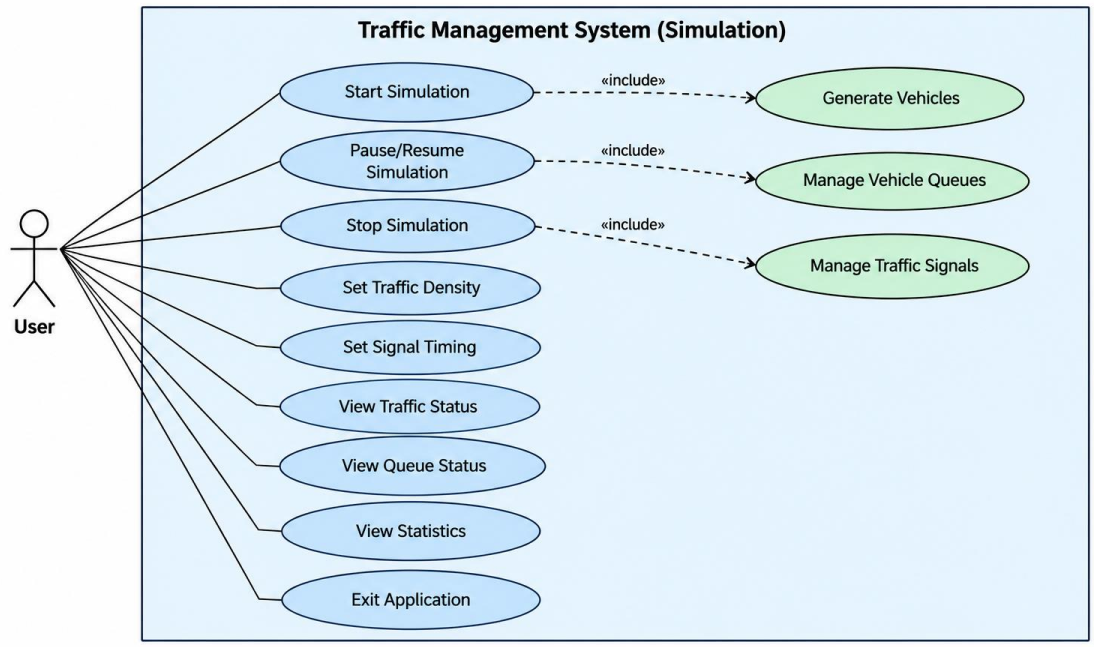
The acceptance criteria will be All high-priority functional requirements (TFS-F-001, 010, 011, 020, 030, 040) implemented and checked with no critical NFR failures, all security requirements (TFS-S-001 to 005) verified, and requirement trace matrix showing all of them being passed. Acceptance test suites: Vehicle Generation, Queue Management, Signal Control, Reporting, Control, Performance, and Security tests.

7. System models and diagrams

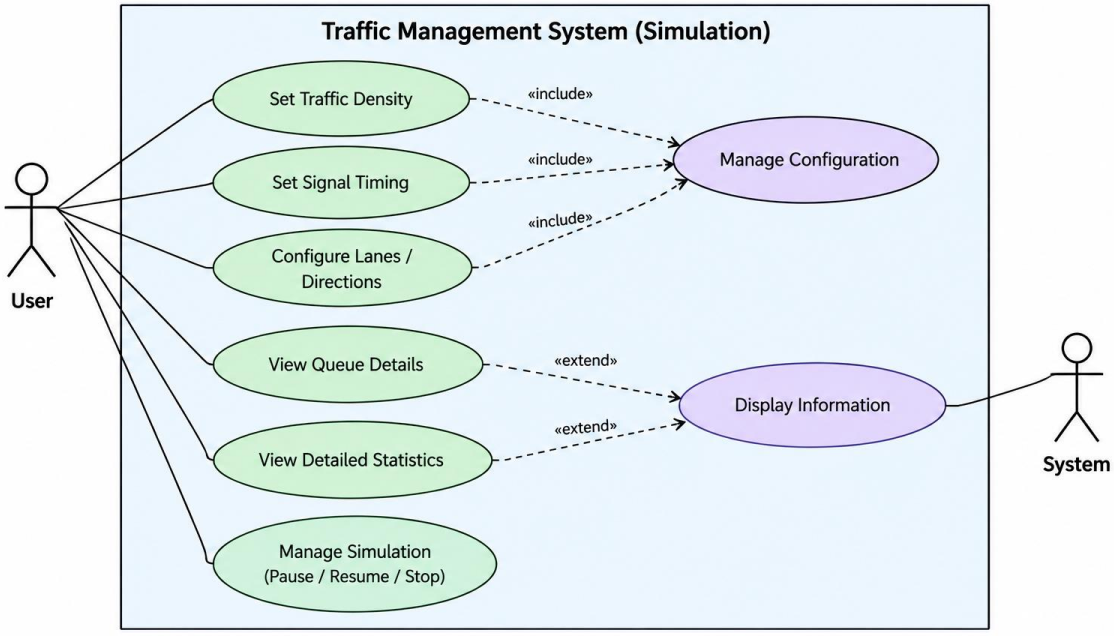
7.1 UML Use-Case diagram

At least 2 UML use case diagrams to be created.

UML Use Case Diagram 1 – Simulation Control and Monitoring



UML Use Case Diagram 2 – Configuration and Information Management



8. Requirements Traceability Matrix (RTM)

Req ID	Description	Source	Design Component	Test Case	Status
TFS-F-001	Generate vehicles at configurable arrival rate	Sec 4.1	VehicleGenerator class	TC-Gen-01	Pending
TFS-F-002	Assign unique ID and timestamp to each vehicle	Sec 4.1	VehicleGenerator / Vehicle class	TC-Gen-02	Pending
TFS-F-010	Enqueue vehicle on red/yellow signal	Sec 4.2	QueueManager class	TC-Queue-01	Pending
TFS-F-011	Dequeue vehicles in FIFO order on green	Sec 4.2	QueueManager class	TC-Queue-02	Pending
TFS-F-012	Enforce max queue capacity & overflow policy	Sec 4.2	QueueManager class	TC-Queue-03	Pending
TFS-F-013	Separate queue per lane/direction	Sec 4.2	QueueManager class	TC-Queue-04	Pending
TFS-F-020	Cycle signal states per configurable durations	Sec 4.3	SignalController class	TC-Sig-01	Pending
TFS-F-021	Independent signal timing per intersection	Sec 4.3	SignalController class	TC-Sig-02	Pending
TFS-F-022	Log every signal state change with timestamp	Sec 4.3	SignalController / Logger	TC-Sig-03	Pending
TFS-F-030	Record wait time & compute average at run end	Sec 4.4	ReportGenerator class	TC-Rep-01	Pending
TFS-F-031	Record maximum queue length observed	Sec 4.4	ReportGenerator class	TC-Rep-02	Pending
TFS-F-032	Output summary report at run end	Sec 4.4	ReportGenerator class	TC-Rep-03	Pending
TFS-F-040	Configure arrival rate, timings, lanes before run	Sec 4.5	ConfigManager class	TC-Cfg-01	Pending
TFS-F-041	Reset and re-run with new parameters	Sec 4.5	SimulationController class	TC-Cfg-02	Pending
TFS-F-042	Save configuration to file for reuse	Sec 4.5	ConfigManager class	TC-Cfg-03	Pending
TFS-NFR-001	Simulate 1000 vehicles/hour without lag	Sec 5	Simulation Engine	TC-NFR-01	Pending

Req ID	Description	Source	Design Component	Test Case	Status
TFS-NFR-002	No vehicle lost/duplicated across operations	Sec 5	QueueManager class	TC-NFR-02	Pending
TFS-NFR-003	Understandable config/report presentation	Sec 5	UI/CLI + ReportGenerator	TC-NFR-03	Pending
TFS-NFR-004	Modular separation of core classes	Sec 5	Overall architecture	TC-NFR-04	Pending
TFS-NFR-005	Cross-platform execution (Win/Linux/macOS)	Sec 5	Runtime/build config	TC-NFR-05	Pending
TFS-S-001	Restrict write access to config files	Sec 5.x	OS file permissions / ConfigManager	TC-Sec-01	Pending
TFS-S-002	Validate/sanitize user-supplied config inputs	Sec 5.x	ConfigManager class	TC-Sec-02	Pending
TFS-S-003	Append-only logs during a run	Sec 5.x	Logger class	TC-Sec-03	Pending
TFS-S-004	Audit entry on configuration changes	Sec 5.x	ConfigManager / Logger	TC-Sec-04	Pending
TFS-S-005	Graceful handling of corrupted/malformed files	Sec 5.x	ConfigManager / FileHandler	TC-Sec-05	Pending